PROJECT CHARTER

KnowledgeVault

Personal Knowledge Base with Semantic Search

MSCS 3999 Capstone, Fall 2026
Clark University

Document Control

Change Record

Date	Author	Version	Change Reference
09/10/2026	Durvank Deorukhkar, Percival Tapera	1.0	Initial draft of Project Charter

Reviewers

Sign Off Date	Reviewer	Position	Sign Off
	Sufyan Almajali	Course Instructor / Project Advisor	
	Dikshya Shrestha	Teaching Assistant	

Distribution

Copy Number	Name
1	Durvank Deorukhkar, Team Member
2	Percival Tapera, Team Member
3	Sufyan Almajali, Instructor / Advisor

Table of Contents

Document Control 3
Table of Contents 4
1 Project Background 5
1.1 Problem/Opportunity Description 5
1.2 Benefits 5
1.3 Goals 5
1.4 Stakeholders and Clients 5
2 Project Scope 7
2.1 Objectives 7
2.2 Deliverables 7
2.3 Out of Scope 8
3 Project Plan 9
3.1 Approach and Methodology 9
3.2 Project Timeline 9
3.3 Success Criteria 10
3.4 Issues and Policy Implications 10
3.5 Risk Management Plan 10
3.6 Service Transition 11
3.7 Options Analysis 12
4 Technical Features 13
5 Project Organization and Staffing 14
6 Project Budget 15

1 Project Background

1.1 Problem/Opportunity Description

Graduate students and knowledge workers accumulate large personal collections of notes, PDFs, and articles across semesters and projects, but the tools they use to store this material (folders, note apps, and cloud drives) only support exact keyword search. As a collection grows past a few hundred items, a person can remember that they read something relevant without remembering the words used to describe it, and keyword search fails them. The result is duplicated research effort, forgotten prior work, and knowledge that becomes effectively unreachable once it is no longer top of mind. The opportunity is to give an individual a personal archive that can be queried the way a person actually thinks about a topic, in plain language, rather than the way the archive happens to be indexed.

1.2 Benefits

KnowledgeVault reduces the time a person spends re-finding material they have already collected, and it surfaces connections between documents that a folder structure cannot represent. Specific benefits include:

- Productivity: faster retrieval of previously read material, reducing duplicated research and re-reading.
- Retention of institutional/personal knowledge: notes and documents stay useful long after they were filed, instead of becoming "write-only" storage.
- Lower cognitive overhead: users no longer need to remember exact filenames, tags, or keywords to relocate a document.
- Course-level benefit: the project gives the team hands-on experience with retrieval-augmented systems, vector search, and full-stack delivery on AWS, directly applicable to the MSCS program's software engineering and applied-AI learning outcomes.

If the project is not undertaken, the team forgoes this applied experience and the capstone deliverable, and the underlying problem (unsearchable personal archives) remains unaddressed for the target users.

1.3 Goals

The goal of this project is to build and deploy a personal knowledge base application that lets a user upload notes, PDFs, and articles and retrieve them later using natural-language, meaning-based search rather than exact keyword matching, with the system automatically generating tags and summaries to make the archive browsable as well as searchable.

1.4 Stakeholders and Clients

Stakeholder	Interest / Role
Team members (Durvank Deorukhkar, Percival Tapera)	Design, build, test, and deliver the application; accountable for all course deliverables.
Course Instructor (Sufyan Almajali)	Acts as project advisor/client proxy; reviews the charter, plan, and deliverables; approves scope and grades milestones.

Teaching Assistant (Dikshya Shrestha)	Supports biweekly status reviews and coordinates feedback between the team and the instructor.
End users (students, researchers, and knowledge workers)	Primary beneficiaries; their workflow (capturing and re-finding notes/PDFs/articles) defines the requirements and success criteria.
Pilot users	A small group of volunteer students/knowledge workers who test upload, tagging, and search during Sprint 3-4 and provide usability and relevance feedback ahead of the final demo.
Cloud & third-party service providers	AWS and the selected vector-database/embedding provider; the deployed service depends on their availability, quotas, and pricing.
Clark University MSCS Program	Interested in the applied demonstration of program learning outcomes and in a deliverable the team can reference professionally.

2 Project Scope

2.1 Objectives

- Requirements and design: define the core user workflows (capture, tag/summarize, search, browse) and the target architecture (FastAPI backend, vector database, React frontend, AWS deployment).
- Document ingestion pipeline: build upload and processing for notes, PDFs, and articles, including text extraction and chunking.
- Semantic search and organization: implement embedding generation, vector search, automatic tagging, and automatic summarization.
- Application delivery: build the React frontend for capture, browsing, and natural-language search, and deploy the full stack to AWS.
- Security and access: ensure uploaded content is stored and retrieved only by its owner, with basic authentication and secure handling of user documents.
- Quality and handoff: test the system end-to-end, and produce documentation, a demo video, and a GitHub repository as the final handoff package for the course.

2.2 Deliverables

Objective 1: Requirements and Design

Project Deliverable	Work Products / Description
Statement of Requirements	Documented user stories for capture, tagging/summarization, search, and browsing.
Reference architecture	Diagram and description of the FastAPI backend, vector database, React frontend, and AWS deployment topology.
Data model	Schema for documents, chunks, embeddings, tags, and summaries.

Objective 2: Core Application (Ingestion, Search, Frontend)

Project Deliverable	Work Products / Description
Upload & ingestion pipeline	Accepts notes/PDF/article uploads; extracts and chunks text for indexing.
Semantic search API	FastAPI endpoints that embed a plain-language query and return ranked, relevant documents from the vector database.
Auto-tagging & summarization	Automatically generated tags and short summaries attached to each uploaded item.
React web application	Capture, browse, and search interface for the end user.

Objective 3: Deployment, Testing, and Handoff

Project Deliverable	Work Products / Description
AWS deployment	Deployed, reachable instance of the application with a working deployment URL.
Test suite & QA results	Functional and search-quality test cases with recorded results.
Documentation & demo video	Project documentation, GitHub repository, and a recorded demonstration for the final capstone submission.

2.3 Out of Scope

- Native mobile applications (iOS/Android): the deliverable is a responsive web application only.
- Multi-tenant/enterprise access-control features (role-based sharing across organizations): this is a personal, single-user archive, not a team platform (that is the separate Enterprise RAG concept the team considered and did not select).
- Integrations with third-party note tools (e.g., Notion, Evernote, Obsidian sync): these are follow-on work if the concept is extended beyond the capstone.
- Offline/local-only operation: the system is designed for cloud deployment on AWS, not an offline desktop client.
- Large-scale performance tuning for archives beyond a single-user demo dataset: the project targets a representative demo corpus, not production-scale load.

3 Project Plan

3.1 Approach and Methodology

The team will follow Agile/Scrum, as required by the course, organized into four two-week sprints between the Project Charter approval and the final capstone submission, with a sprint review (status report) due at the end of each sprint. Durvank and Percival will split work primarily along backend/ML (ingestion, embeddings, search, tagging/summarization) and frontend/deployment (React UI, AWS infrastructure) lines, pairing on integration points, and will rotate the Scrum Master role for running standups and sprint planning.

Tools and stack: FastAPI (backend/API), a vector database for semantic search (evaluated in Section 3.7), React (frontend), and AWS for deployment (evaluated for compute/hosting). GitHub will be used for source control and issue tracking, and the team will hold weekly internal check-ins in addition to the biweekly instructor/advisor meetings required by the syllabus.

3.2 Project Timeline

ID	Task Name	Owner	Start	Finish	Duration
1	Idea selection & team formation	Both	08/27/26	09/03/26	1 week
2	Requirements gathering & Project Charter	Both	09/03/26	09/17/26	2 weeks
3	Architecture design, vector DB selection & vision/roadmap	Durvank	09/17/26	09/24/26	1 week
4	Sprint 1: ingestion pipeline & data model	Durvank	09/24/26	10/01/26	1 week
5	Sprint 1 review / status report #1	Both	10/01/26	10/01/26	Milestone
6	Sprint 2: embeddings, vector search & tagging/summarization	Durvank	10/01/26	10/15/26	2 weeks
7	Sprint 2 review / status report #2	Both	10/15/26	10/15/26	Milestone
8	Sprint 3: React frontend (capture, browse, search UI)	Percival	10/15/26	10/29/26	2 weeks
9	Pilot testing session with volunteer users	Percival	10/22/26	10/29/26	1 week
10	Sprint 3 review / status report #3	Both	10/29/26	10/29/26	Milestone
11	Sprint 4: AWS deployment, auth/security, integration	Percival	10/29/26	11/12/26	2 weeks
12	Sprint 4 review / status report #4	Both	11/12/26	11/12/26	Milestone
13	End-to-end, accessibility & security testing; bug fixing	Both	11/12/26	11/19/26	1 week
14	Documentation, demo video & deliverables check-in	Both	11/19/26	12/03/26	2 weeks

15	Final polish & presentation preparation	Both	12/03/26	12/10/26	1 week
16	Final presentation & deliverable submission	Both	12/10/26	12/10/26	Milestone

3.3 Success Criteria

- A user can sign in, upload at least three supported document types (note, PDF, article text), and see each item indexed with clear processing status feedback.
- For a curated test set of at least 20 documents and 15 representative plain-language queries, at least 80% of queries return a relevant source within the top 5 results, judged against a predefined expected-source key.
- In pilot testing (Task 9), at least 80% of pilot participants can upload an item and locate a previously saved fact using a plain-language query without facilitator help.
- The application is reachable at a deployed AWS URL with core upload/search/browse flows working end-to-end without manual intervention, and all critical/high-severity defects are resolved or documented with a workaround before final submission.
- All four sprint status reports, the project charter, the vision/roadmap, and the final documentation/demo/GitHub/presentation deliverables are submitted on schedule, per the syllabus grading breakdown.

3.4 Issues and Policy Implications

- Dependency on course schedule: the charter, roadmap, and sprint reviews are tied to fixed syllabus dates (e.g., charter due Week 3-4), so scope must fit the available weeks.
- Data handling and privacy: uploaded documents may contain personal or sensitive notes; the team will use only authorized, non-sensitive test/demo content, isolate each user's documents behind authentication, and publish a short privacy notice describing what is stored, how long, and how a user can delete it.
- Accessibility: core pages will follow WCAG 2.1 AA-style practices (keyboard navigation, visible focus, labeled controls, readable contrast) consistent with the course's emphasis on usable delivery.
- AI-assisted work: per the syllabus Generative AI Use Statement, any AI tool used in producing deliverables must be disclosed by tool name and contribution in the final report and presentation.
- Third-party API/service dependency: the vector database and any hosted embedding service introduce external dependencies outside the team's direct control.

3.5 Risk Management Plan

Risks are identified during sprint planning and reviewed at each biweekly status check-in with the instructor/advisor; new risks are logged and escalated to the advisor if they threaten a milestone date.

Risk Factor	Probability (H-M-L)	Impact (H-M-L)	Risk Management Action
Semantic search returns poor-quality matches on real documents	M	H	Start embedding/retrieval work early (Sprint 2); test against a small labeled query set and tune

			chunking/embedding approach before building on top of it.
AWS deployment/configuration issues near the deadline	M	H	Deploy a minimal version early (Sprint 3-4) rather than only at the end; keep infrastructure-as-code/scripts so deployment is repeatable.
Team member availability conflicts (coursework/work commitments)	M	M	Weekly check-ins to redistribute tasks; maintain a shared backlog so either member can pick up blocked work.
Scope creep beyond the two-person team's capacity	M	M	Enforce the Out of Scope list (Section 2.3); new requests are logged for a future phase rather than added mid-sprint.
Vector database / hosting cost overruns	L	M	Use free-tier or low-cost managed options (Section 3.7) and monitor usage against the budget in Section 6.
Sensitive or unauthorized content is uploaded during testing	M	H	Use only approved, non-sensitive test data; validate file types; restrict access per user; support deletion; suspend ingestion if a privacy concern occurs.
Security weakness or an exposed secret (API key, credential)	L	H	Use environment variables and secret scanning, least-privilege AWS roles, and a pre-release security check; rotate credentials and redeploy immediately if one is discovered.

3.6 Service Transition

KnowledgeVault is an academic capstone deliverable rather than an ongoing ITS-operated service, so "service transition" here means handing the project off cleanly at the end of the course rather than into a production support organization:

- Support after handoff: no dedicated support desk exists; the GitHub repository, README, and setup documentation are written so a future maintainer (including the team itself, if continued) can stand the system up independently.
- Impact mitigation: because there are no live end users beyond the demo/evaluation audience, there is no user-facing cutover to manage; the deployed AWS instance is documented with its configuration so it can be redeployed or retired cleanly.
- Transitional support: during the final two weeks (Weeks 13-15), the team focuses on stabilizing the deployed instance so it remains available for the instructor's review, demo recording, and live presentation.
- Artifacts delivered at transition: source code (GitHub), deployment URL, architecture and requirements documentation, test results, and a demo video, per the course's Capstone Project Deliverables requirement.

3.7 Options Analysis

The team compared the following approaches before committing to a custom-built application:

Option	Advantages	Limitations	Decision
Do nothing: keep using folders/existing note apps	No implementation effort or cost.	Only exact keyword search; the duplication and lost-knowledge problem in Section 1.1 remains unsolved.	Rejected
Extend an existing note tool's built-in search	Lower build effort; familiar tool for users.	Most consumer note apps don't expose the APIs needed for custom embedding-based search; infeasible in the course timeline.	Rejected
Adopt a commercial personal-knowledge-management tool	Fastest route to a mature feature set.	Limited control over data handling and architecture; does not meet the course's requirement to build and demonstrate the system.	Rejected
Custom-built KnowledgeVault (FastAPI + vector database + React)	Full control over ingestion, tagging/summarization, and search quality; directly demonstrates the required architecture and semantic-search approach.	Requires careful scope control and the team's own operational safeguards (security, cost limits).	Selected

Within the selected approach, the team will still evaluate open-source vs. managed vector database options (self-hosted vs. a fully managed vector search service) in Sprint 1-2, choosing based on ease of FastAPI integration, cost within the Section 6 budget, and setup time.

4 Technical Features

KnowledgeVault is a full-stack web application with a FastAPI backend, a vector database for semantic search, and a React frontend, deployed on AWS. At a broad level, the application provides:

- Document capture: upload of notes, PDFs, and articles, with automatic text extraction from each format.
- Automatic organization: system-generated tags and short summaries attached to each uploaded item without manual effort from the user.
- Semantic (meaning-based) search: a plain-language search box that embeds the query and matches it against the vector database, returning relevant results even when wording differs from the source text.
- Browsing interface: a React frontend for reviewing the archive by tag, recency, or search relevance, and for reading/managing individual items.
- Identity and access: authenticated accounts with user-level ownership, so each person's uploads and searches are isolated from other users.
- Security and privacy: encryption in transit, no hard-coded secrets, and log/data-retention practices consistent with the privacy notice in Section 3.4.
- Cloud deployment: hosted on AWS, with the backend, frontend, and vector database components deployed as a reachable application rather than a local-only tool.

Specific product/service selections (for example, the exact vector database and embedding provider) will be finalized during the architecture and options-analysis work in Sprint 1, based on cost, ease of FastAPI integration, and setup time (see Section 3.7).

5 Project Organization and Staffing

As a two-person capstone team, Durvank and Percival hold the project management and delivery roles jointly, with the course instructor acting as executive/project sponsor and advisor. The Scrum Master role rotates between the two team members each sprint to share planning and reporting responsibility. Escalations beyond the team's ability to resolve (e.g., scope changes, timeline risk) are raised to the instructor at the biweekly advisor meeting or by email, per the syllabus.

Role	Names & Contact Information	Responsibilities	Time
Executive Sponsor	Sufyan Almajali (SAlmajali@clarku.edu)	Serve as ultimate course-level authority for the project Approve the charter and major scope changes	Advisory
Project Sponsor / Advisor	Sufyan Almajali (SAlmajali@clarku.edu); TA: Dikshya Shrestha (DikShrestha@clarku.edu)	Review biweekly status reports and sprint outcomes Provide feedback and approve deliverables and change requests	Biweekly
Project Manager / Scrum Master (rotating)	Durvank Deorukhkar and Percival Tapera (alternating by sprint)	Run sprint planning and standups Track the backlog and prepare status reports Escalate risks and blockers to the advisor	Shared
Backend / ML Engineer	Durvank Deorukhkar	Build the FastAPI backend and ingestion pipeline Implement embeddings, vector search, tagging and summarization	~50%
Frontend / DevOps Engineer	Percival Tapera	Build the React frontend (capture, browse, search) Own AWS deployment, environment configuration, and CI	~50%
Advisors and Resources	Clark University MSCS faculty; Clark IT/library resources as needed	Provide guidance on methodology and technical review Available for consultation, not day-to-day execution	As needed

6 Project Budget

As a student capstone project, actual cash outlay is intentionally minimal and relies on free tiers and low-cost cloud resources wherever possible. No real payroll exists, but an in-kind labor estimate is included below (at a nominal planning rate, not a cash cost) so the full effort behind the project is visible, consistent with the template's guidance to account for personnel costs.

Budget Item	Description	Budgeted Cost
One-Time Costs		
Team labor (in-kind, not a cash cost)	Approx. 320 combined hours across both members over the semester, at a nominal planning rate of $25/hr, shown for effort visibility only.	$8,000 (in-kind)
Domain / project setup	Optional custom domain or project registration for the deployed demo.	$15
Development tooling	GitHub (free tier), design/diagramming tools (free tier).	$0
Total One-Time Cash Costs	Excludes in-kind labor.	$15
Ongoing Costs		
AWS compute & storage	Backend hosting, storage for uploaded documents, and vector database hosting (free-tier eligible for the semester's demo workload).	$0–$25/mo
Embedding / LLM API usage	API calls for embeddings and automatic tagging/summarization during development and demo use.	$0–$20/mo
Total Ongoing Costs		$0–$45/mo

Costs will be monitored against AWS and API free-tier limits throughout the semester; if usage approaches paid thresholds, the team will scale back demo data volume rather than exceed the budget above.